

**A REPORT**  
**ON**  
**RESOURCE-CONSTRAINED IMPLEMENTATIONS OF**  
**AI ALGORITHMS**

**BY**

**RAJAT SRIVASTAVA**

**2022B1A81375G**

**SHIVAM RAJ**

**2022A8PS0397P**

**AT**

**CSIR-CEERI PILANI**

**A Practice School-I Station of**

**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI**

**(JUNE,2024)**

**A REPORT  
ON  
RESOURCE-CONSTRAINED IMPLEMENTATIONS OF  
AI ALGORITHMS**

Prepared in fulfillment of the Practice School-I Course Nos. BITS C221/BITS C231/BITS C241

**BY**

**RAJAT SRIVASTAVA**

**2022B1A81375G**

**SHIVAM RAJ**

**2022A8PS0397P**

**AT**

**CSIR-CEERI PILANI**

**A Practice School-I Station of**

**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI**

**(JUNE,2024)**

## Acknowledgements

We would like to extend our heartfelt gratitude to the individuals who have played a pivotal role in our journey during the practice school at BITS Pilani for the project titled "Resource Constrained Implementations of AI Algorithms."

First and foremost, we express our sincere appreciation to our esteemed faculty in charges,

Dr. Pankaj Arora and Dr. Meetha Shenoy. Your unwavering support, guidance, and encouragement have been instrumental in our learning and development throughout this project. Your dedication to our success and your insightful advice have been invaluable to us.

We are also deeply grateful to our project instructors, Dr. Sumeet Saurav and Dr. Ravi Saini. Your expertise, patience, and mentorship have been crucial in navigating the complexities of implementing AI algorithms within resource constraints. Your constructive feedback and willingness to share your knowledge have significantly contributed to the progress and success of our project.

Additionally, we would like to thank the Practice School Division for providing us with this invaluable opportunity. Your commitment to fostering an environment of practical learning and professional development has greatly enhanced our educational experience. The support and resources you have provided have been fundamental in the successful execution of our project.

Thank you all for your unwavering support and guidance. Your contributions have made a lasting impact on our academic and professional growth, and we are truly grateful for the opportunity to learn from such distinguished educators and mentors.

**Format of an Abstract Sheet**  
**BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI**  
**(RAJASTHAN)**  
**Practice School Division**

Station: ...CEERI PILANI.....Centre .....Pilani.....

Duration.....2 months ..... Date of Start.....28/05/2024.....

Date of Submission.....21/07/2024.....

Title of the Project:

|                                    |                  |               |
|------------------------------------|------------------|---------------|
| ID No./Name(s)/                    | RAJAT SRIVASTAVA | 2022B1A81375G |
| Discipline(s)/of<br>the student(s) | SHIVAM RAJ       | 2022A8PS0397P |

|                                                    |                                                                         |
|----------------------------------------------------|-------------------------------------------------------------------------|
| Name(s) and<br>designation(s)<br>of the expert(s): | Dr. Sumeet Saurav, Senior Scientist<br>Dr. Ravi Saini, Senior Scientist |
|----------------------------------------------------|-------------------------------------------------------------------------|

|                               |                                       |
|-------------------------------|---------------------------------------|
| Name(s) of<br>the PS Faculty: | Dr. Pankaj Arora<br>Dr. Meetha Shenoy |
|-------------------------------|---------------------------------------|

Key Words: Human Activity Recognition, TinyML, Arduino

Project Areas: **Tiny ML and Wearable Sensors:** Integrating AI for applications involving wearable devices and sensor data, such as human activity recognition.

Abstract: Deploying artificial intelligence (AI) algorithms in resource-constrained environments poses challenges and opportunities across sectors. This abstract explores optimizing AI for limited computational and power resources, focusing on Tiny Machine Learning (TinyML) and Arduino. These technologies democratize AI capabilities, enabling real-time, energy-efficient applications at the edge. Integrating AI with edge devices illustrates practical implementations and transformative impacts in various fields.

Signature(s) of Student(s)  
Date:21/07/24



Signature of PS Faculty  
Date:21/07/24

# TABLE OF CONTENTS

1. Cover
2. Title Page
3. Acknowledgements
4. Abstract Sheet
5. Table of Contents
6. Introduction
7. Main Text
  - Background on CNN
  - Techniques for Resource-Constrained Implementations
  - MobileNets in Edge computing example
  - Literature Review
  - Work Progress
- Post Midsem-----
8. Work Done
9. Results & Conclusions
10. Appendices
- 11.. References
12. Glossary

## INTRODUCTION

In recent years, artificial intelligence (AI) has permeated various sectors, driving innovation and enhancing capabilities in ways previously thought unattainable. One particularly exciting application is human activity recognition (HAR), which has gained significant attention due to its potential in healthcare, fitness tracking, and human-computer interaction. This report delves into the implementation of a HAR system using the Arduino Nano 33 BLE Sense, a resource-constrained edge device, in conjunction with TensorFlow Lite for efficient on-device inference.

The focus of our research is the development of a real-time HAR system capable of classifying six distinct human activities using the WISDM (Wireless Sensor Data Mining) dataset. This dataset, which contains accelerometer data from smartphones, provides a robust foundation for training our model to recognize common activities such as walking, jogging, sitting, standing, climbing stairs, and descending stairs.

Our implementation leverages the concept of Tiny Machine Learning (TinyML), which refers to the deployment of machine learning models on microcontrollers and other ultra-low-power hardware. By utilizing TensorFlow Lite, a lightweight version of TensorFlow designed for mobile and embedded devices, we can run efficient inference on the Arduino Nano 33 BLE Sense. This approach allows us to achieve real-time activity recognition while adhering to the strict resource limitations of edge devices.

The Arduino Nano 33 BLE Sense is particularly well-suited for this task due to its integrated sensors, including an accelerometer and gyroscope, which can capture the motion data necessary for activity recognition. Its Bluetooth Low Energy (BLE) capability also enables wireless communication, making it ideal for wearable or portable HAR applications.

This report explores the methodologies and technologies that enable HAR in resource-constrained environments. We will examine the entire process, from data collection and preprocessing using the WISDM dataset to model architecture design, training, and deployment on the Arduino Nano 33 BLE Sense. The challenges encountered during development and the solutions employed to overcome them will be discussed in detail.

Our analysis will cover the performance of the system in terms of accuracy, latency, and power consumption, providing insights into the feasibility and effectiveness of implementing HAR on resource-constrained devices. Furthermore, we will explore the potential applications of this technology and its implications for the future of wearable devices and ubiquitous computing.

By showcasing the capabilities of TinyML and edge AI in the context of human activity recognition, this report aims to contribute to the growing body of knowledge in this field and inspire further research and development in resource-efficient AI applications. The successful implementation of a real-time HAR system on a small, low-power device like the Arduino Nano 33 BLE Sense demonstrates the potential for widespread deployment of intelligent, context-aware systems in everyday objects and environments.

## MAIN TEXT

Convolutional Neural Networks (CNNs) are a class of deep learning algorithms particularly well-suited for processing data with a grid-like topology, such as images. CNNs are composed of layers that transform the input image into a representation that encodes the important features, allowing for tasks such as image classification, object detection, and segmentation.

A typical CNN architecture consists of several key layers:

- **Convolutional Layers:** These layers apply a set of learnable filters to the input to extract local features.
- **Pooling Layers:** These layers downsample the spatial dimensions of the feature maps, reducing the computational load and promoting spatial invariance.
- **Fully Connected Layers:** These layers act as the final classifier, using the features extracted by the convolutional layers.

### *Implementing CNNs in Resource-Constrained Environments*

When deploying CNNs on devices with limited computational resources, such as edge devices or embedded systems, several techniques and optimizations can be applied to ensure efficient operation without sacrificing significant accuracy.

#### *Techniques for Resource-Constrained Implementations*

##### 1. **Quantization**

Quantization reduces the precision of the numbers used to represent the model's weights and activations, typically from 32-bit floating-point to 8-bit integer. This reduction decreases the memory footprint and increases the inference speed of the model. Quantization can be applied during training (quantization-aware training) or after training (post-training quantization).

- **Uniform Quantization:** All weights and activations are uniformly quantized to a fixed number of bits.
- **Non-Uniform Quantization:** Different parts of the network are quantized with different precisions based on their sensitivity to quantization errors.
- **Mixed-Precision Quantization:** A hybrid approach where some layers use higher precision than others

##### 2. **Pruning**

Pruning involves removing less important connections (weights) from the neural network, resulting in a sparser model. This can significantly reduce the model size and computational cost while maintaining a similar level of performance.

- **Magnitude-based Pruning:** Weights with magnitudes below a certain threshold are pruned.
- **Gradient-based Pruning:** Weights that contribute the least to the loss reduction during training are pruned.

##### 3. **Efficient Neural Network Architectures**

Designing networks with inherently efficient architectures can reduce the computational burden. Some popular efficient architectures include:

- **DepthWise Separable Convolutions:** These convolutions factorize a standard convolution into a depthwise convolution and a pointwise convolution, significantly reducing the number of parameters and computations.
- **Inverted Residual Blocks:** Used in MobileNetV2, these blocks combine depthwise separable convolutions with residual connections to maintain accuracy while reducing the model size.
- **MobileNets, SqueezeNets, EfficientNets:** These are specifically designed for resource-constrained environments, offering a good balance between model size and accuracy.

#### 4. Neural Architecture Search (NAS)

NAS techniques automate the design of neural network architectures to find the most efficient model for a given task and resource constraint. This approach uses algorithms to search the space of possible architectures and select the best-performing ones.

#### 5. Hardware Acceleration

Leveraging specialized hardware features can optimize the execution of CNNs on resource-constrained devices:

- **SIMD Instructions and Vectorization:** Single Instruction, Multiple Data (SIMD) instructions perform the same operation on multiple data points simultaneously, enhancing parallelism.
- **Kernel Optimization Techniques:** Techniques such as tiling (breaking down computation into smaller, manageable blocks) and loop unrolling (reducing the overhead of loop control) can improve the efficiency of CNN computations.

#### 6. On-Device Learning and Inference

- **Federated Learning:** This approach involves training models across multiple decentralized devices while keeping the data localized, enhancing privacy and reducing communication overhead.
- **Incremental Learning:** Techniques that enable the model to learn from new data incrementally without retraining from scratch.

#### 7. Model Partitioning and Offloading

- **Split Computation Strategies:** The model's computations are divided between the device and the cloud to balance the load.
- **Communication Overhead Management:** Techniques to minimize the data transfer between the device and the cloud.

#### 8. Benchmarking and Evaluation

Evaluating the performance of CNNs in resource-constrained environments involves considering metrics like accuracy, latency, and memory footprint. Frameworks such as MLPerf and AI Benchmark provide standardized ways to measure these metrics and compare different models and approaches.



### *Practical Example: MobileNets in Edge Computing*

MobileNets are a family of efficient models designed for mobile and embedded vision applications. They use depthwise separable convolutions to significantly reduce the number of parameters and operations compared to traditional convolutions, making them well-suited for resource-constrained devices.

#### **Implementation Steps:**

1. **Model Design:** Choose a MobileNet variant (e.g., MobileNetV2) and configure it for the specific task.
2. **Quantization:** Apply quantization-aware training to prepare the model for deployment.
3. **Pruning:** Prune the model to further reduce its size and computational requirements.
4. **Deployment:** Use a framework like TensorFlow Lite to convert and deploy the model on the edge device.
5. **Optimization:** Utilize hardware acceleration features of the device, such as SIMD instructions, to optimize inference performance.
6. **Evaluation:** Benchmark the model's performance using accuracy, latency, and memory footprint metrics.

## **LITERATURE REVIEW**

*'TCN-attention-HAR: human activity recognition based on attention mechanism time convolutional network'*

#### **Current State of the Art:**

1. **Hybrid Deep Learning Models:** Recent studies have combined CNNs and RNNs (such as LSTM and GRU) to leverage both spatial and temporal features from sensor data. This hybrid approach has been shown to enhance accuracy in activity recognition tasks.
  - o Example: A multichannel CNN-GRU model proposed by Lu et al. (2022) demonstrated improved performance in recognizing complex activities.
2. **Attention Mechanisms:** To handle noisy data and focus on the most relevant features, attention mechanisms have been integrated into deep learning models. These mechanisms help improve the precision of activity recognition.
  - o Example: Pan et al. (2022) utilized GRU with dual attentions, resulting in significant performance improvements.
3. **Data Augmentation and Synthetic Data:** Data augmentation techniques and the use of synthetic data have been explored to address the challenge of limited annotated datasets. These methods aim to create a more diverse and balanced training set.
  - o Example: Techniques such as oversampling and synthetic data generation have been employed to enhance model training, as discussed by Azmat and Jalal (2021).

#### **Gaps in the Research:**

1. **Domain Shift Problem:** Despite the potential of synthetic data, models often struggle with the domain shift when transitioning from synthetic training data to real-world testing data. This gap highlights the need for robust domain adaptation techniques.
2. **Realism in Synthetic Data:** The current methods of generating synthetic data may lack the realism required for effective training, leading to reduced model performance when applied to real-world scenarios.
3. **Integration of Multiple Sensor Modalities:** While most studies focus on single sensor data, integrating multiple sensor modalities could provide richer information and improve recognition accuracy. However, this area remains underexplored.

#### **Objectives:**

1. **Develop Robust HAR Models:** Create hybrid deep learning models combining CNNs and RNNs (LSTM/GRU) to effectively recognize human activities using wearable sensor data.

2. **Incorporate Attention Mechanisms:** Integrate attention mechanisms into the models to enhance focus on relevant features and improve performance in noisy data environments.
3. **Enhance Data Augmentation Techniques:** Develop advanced data augmentation methods and explore the use of synthetic data to increase the diversity and robustness of the training dataset.
4. **Address Domain Shift:** Implement domain adaptation techniques to bridge the gap between synthetic training data and real-world application.

**Scope:** The scope of this work encompasses the development, evaluation, and optimization of deep learning models for HAR using wearable sensors. It includes:

- Collection and preprocessing of sensor data.
- Designing hybrid CNN-RNN architectures.
- Incorporating attention mechanisms.
- Employing advanced data augmentation and synthetic data generation techniques.
- Testing and validating models on benchmark datasets such as USC-HAD and PAMAP2.

### *Work Progress Achieved Till Now*

Our journey into deep learning began with Andrew Ng's Deep Learning Specialization on Coursera. This comprehensive series of courses provided a thorough grounding in neural networks, convolutional networks, sequence models, and advanced techniques essential for modern AI applications. Key highlights from each module include:

- **Neural Networks and Deep Learning:** We acquired fundamental knowledge of neural networks, including their architecture, backpropagation algorithm, and principles of deep learning.
- **Improving Deep Neural Networks: Hyperparameter tuning, Regularization, and Optimization:** Techniques such as hyperparameter tuning, regularization (L2 and dropout), and optimization algorithms (Adam, RMSprop) were explored in depth, crucial for improving model performance and preventing overfitting.
- **Structuring Machine Learning Projects:** This module provided strategies for managing machine learning projects effectively, covering topics like data organization, error analysis, and model evaluation.
- **Convolutional Neural Networks:** We studied CNNs extensively, learning about their architecture, applications in image analysis (classification, detection, segmentation), transfer learning, and data augmentation techniques.
- **Sequence Models:** This module focused on RNNs, LSTM, and GRU networks for sequential data analysis, including applications in speech recognition, machine translation, and sentiment analysis.

### *Coursera's "Getting Started with Machine Learning at the Edge on Arm"*

Simultaneously, we enrolled in "Getting Started with Machine Learning at the Edge on Arm" on Coursera, which focused on deploying machine learning models on resource-constrained edge devices powered by ARM processors. Key takeaways from this course include:

- **Edge AI Fundamentals:** Understanding the unique challenges and constraints of deploying AI models on edge devices like microcontrollers and embedded systems.
- **Implementation with Accelerometers:** Hands-on experience in using accelerometer data to categorize sporting activities, covering data preprocessing, model training, and deployment for real-time inference.

### *NVIDIA Jetson, MorphNet, and TensorRT*

Additionally, we explored advanced tools and technologies essential for efficient AI deployment:

- **NVIDIA Jetson:** Familiarization with NVIDIA Jetson platforms, which provide high-performance computing capabilities suitable for edge AI applications. This included practical exercises in optimizing model performance and achieving real-time inference.
- **MorphNet:** Exploration of MorphNet, a neural network optimization technique that adjusts network structure during training to optimize both accuracy and computational efficiency.
- **TensorRT:** Learning about NVIDIA's TensorRT, a high-performance deep learning inference library that enables optimizations such as layer fusion, precision calibration, and GPU-accelerated inference.

### *TinyML and Arduino*

We also delved into Tiny Machine Learning (TinyML), which focuses on deploying machine learning models on microcontrollers and other ultra-low-power devices like Arduino. Key aspects covered include:

- **TinyML Fundamentals:** Understanding the principles of TinyML, which allows AI models to run efficiently on devices with minimal computational resources and power consumption.
- **Arduino Integration:** We will gain practical experience in integrating machine learning models with Arduino platforms, enabling applications such as sensor data processing, predictive maintenance, and environmental monitoring.
- **Will be working on Accelerometer Data to categorize sporting activities**

### **Future Work:**

- Further refine the hybrid models and attention mechanisms.
- Enhance the realism of synthetic data using advanced transformation techniques.
- Develop and implement domain adaptation strategies to improve real-world applicability.
- Expand the scope to include multiple sensor modalities for richer data representation and improved recognition accuracy.

## CONCLUSION

The integration of artificial intelligence (AI) algorithms into resource-constrained environments presents both challenges and transformative opportunities across various sectors. This report has underscored the importance of optimizing AI models to operate effectively on limited hardware and power resources. It highlighted the emergence of Tiny Machine Learning (TinyML) as a pivotal advancement, enabling efficient deployment of machine learning models on microcontrollers and other ultra-low-power devices. By leveraging TinyML, AI capabilities can reach new frontiers, from remote areas with minimal infrastructure to portable and wearable devices requiring stringent power efficiency.

Moreover, the implementation of Convolutional Neural Networks (CNNs) in resource-constrained environments was explored, emphasizing techniques such as quantization, pruning, efficient network architectures, and hardware acceleration. These methodologies ensure that CNNs can deliver high-performance inference while adhering to constraints on computational resources and power consumption. Practical examples like MobileNets illustrated the effective deployment of efficient CNN architectures in edge computing scenarios, demonstrating significant reductions in model size and computational requirements without compromising accuracy.

## RECOMMENDATIONS

Based on the literature review and analysis conducted in this report, several recommendations are proposed for advancing research and practical implementations in AI for resource-constrained environments:

1. **Further Research on Domain Adaptation:** Addressing the domain shift problem between synthetic training data and real-world applications remains a critical challenge. Future research should focus on developing robust domain adaptation techniques to enhance the generalizability of AI models across different environments.
2. **Enhancement of Synthetic Data Realism:** Improving the realism of synthetic data generation methods is essential for effective model training. Advanced techniques should be explored to create synthetic datasets that closely mimic real-world scenarios, thereby improving the performance of AI models when deployed in practical applications.
3. **Integration of Multiple Sensor Modalities:** While current studies primarily focus on single-sensor data, integrating multiple sensor modalities could provide richer information for activity recognition and other AI applications. Research efforts should investigate fusion techniques that leverage diverse sensor inputs to enhance accuracy and reliability.
4. **Standardization of Benchmarking Metrics:** Establishing standardized metrics and evaluation frameworks (such as MLPerf and AI Benchmark) for measuring the performance of AI models in resource-constrained environments is crucial. This will facilitate objective comparisons between different models and approaches, guiding researchers and developers in selecting optimal solutions.
5. **Exploration of Federated Learning and On-Device AI:** With increasing concerns over data privacy and communication overhead, exploring federated learning and on-device AI techniques is recommended. These approaches enable distributed model training and inference while preserving data privacy and reducing reliance on centralized infrastructure.

## APPENDICES

### *Appendix A: List of Acronyms*

- CNN: Convolutional Neural Network
- RNN: Recurrent Neural Network
- LSTM: Long Short-Term Memory
- GRU: Gated Recurrent Unit
- Bi-LSTM: Bi-Directional LSTM
- IMU: Inertial Measurement Unit
- GANs: Generative Adversarial Networks
- mAP: Mean Average Precision
- TIP: Threat Image Projection
- HMMs: Hidden Markov Models

## Appendix B: Example Architectures

### Example 1: Multichannel CNN-GRU Model

- **CNN Layers:**
  - Convolutional layers for feature extraction from sensor data.
  - Pooling layers to reduce spatial dimensions.
- **GRU Layer:**
  - Bidirectional GRU for temporal sequence processing.
- **Output Layer:**
  - Softmax layer for activity classification.

### Example 2: Ensemble Model

- **Base Models:**
  - Multiple CNN architectures (e.g., ResNet, VGG) trained on different subsets of data.
- **Ensemble Method:**
  - Voting or averaging predictions from individual models.
- **Output Layer:**
  - Final decision based on aggregated predictions.

## Appendix C: Data Augmentation Techniques

- **Translation:** Shifting data points horizontally or vertically.
- **Rotation:** Rotating sensor data around a central point.
- **Scaling:** Adjusting the magnitude of data points uniformly.
- **Flipping:** Reflecting data points across an axis.
- **Noise Addition:** Introducing random noise to simulate real-world variability.

## Appendix D: Sample Experimental Results

- **Model Performance Metrics:**
  - Accuracy, precision, recall, F1-score.
- **Comparison Tables:**
  - Performance of different models on benchmark datasets.
- **Graphs and Plots:**
  - Learning curves, confusion matrices, ROC curves.

## Appendix E: Code Snippets

```
python
Copy code
# Example code for a basic CNN model in HAR
import tensorflow as tf
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Conv1D(64, 3, activation='relu', input_shape=(100, 3)),
    layers.MaxPooling1D(2),
    layers.Conv1D(128, 3, activation='relu'),
    layers.GlobalAveragePooling1D(),
    layers.Dense(6, activation='softmax')
])
```

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
model.summary()
```

#### *Appendix F: Detailed Experimental Setup*

- **Dataset Description:**
  - Source, size, classes, data preprocessing steps.
- **Training Configuration:**
  - Hyperparameters (learning rate, batch size, epochs).
- **Hardware and Software Specifications:**
  - GPU/CPU used, TensorFlow/PyTorch version, operating system.

These appendices provide additional information and resources related to the study of Human Activity Recognition using wearable sensors and deep learning techniques. They aim to enhance understanding and facilitate further research in this domain.

## REFERENCES

1. Azmat, U., & Jalal, A. (2021). Smartphone inertial sensors for human locomotion activity recognition based on template matching and codebook generation. In *2021 International Conference on Communication Technologies (ComTech)*, Rawalpindi, Pakistan, 109–114.
2. Bhattacharya, D., Sharma, D., Kim, W., Ijaz, M. F., & Singh, P. K. (2022). Ensem-HAR: An ensemble deep learning model for smartphone sensor-based human activity recognition for measurement of elderly health monitoring. *Biosensors*, 12(6), 393.
3. Dua, N., Singh, S. N., & Semwal, V. B. (2021). Multi-input CNN-GRU based human activity recognition using wearable sensors. *Computing*, 103(7), 1461–1478.
4. Jalal, A., Batool, M., & Kim, K. (2020). Sustainable wearable system: Human behavior modeling for life-logging activities using K-Ary tree hashing classifier. *Sustainability*, 12(10324).
5. Jalal, A., Kamal, S., & Kim, D. (2015). Depth silhouettes context: A new robust feature for human tracking and activity recognition based on embedded HMMs. In *2015 12th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*, Goyangi, Korea (South), 294–299.
6. Jalal, A., Kamal, S., Farooq, A., & Kim, D. (2015). A spatiotemporal motion variation features extraction approach for human tracking and pose-based action recognition. In *2015 International Conference on Informatics, Electronics & Vision (ICIEV)*, Fukuoka, Japan, 1–6.
7. Kwapisz, J. R., Weiss, G. M., & Moore, S. A. (2011). Activity recognition using cell phone accelerometers. *SIGKDD Explor. Newsl.*, 12, 74–82.
8. Lu, L., Zhang, C., Cao, K., Deng, T., & Yang, Q. (2022). A multichannel CNN-GRU model for human activity recognition. *IEEE Access*, 10, 66797–66810.
9. Nadeem, A., Jalal, A., & Kim, K. (2021). Automatic human posture estimation for sport activity recognition with robust body parts detection and entropy markov model. *Multimed. Tools Appl.*, 80, 21465–21498.
10. Pan, J., Zhengxin, H., Yin, S., & Li, M. (2022). GRU with dual attentions for sensor-based human activity recognition. *Electronics*, 11(11), 1797.
11. Reiss, A., & Stricker, D. (2012). Introducing a new benchmarked dataset for activity monitoring. In *Proceedings of the 2012 16th International Symposium on Wearable Computers*, Newcastle, UK, 108–109.
12. Semwal, V. B., et al. (2023). Gait reference trajectory generation at different walking speeds using LSTM and CNN. *Multimed. Tools Appl.*, 82, 33401–33419.
13. Yin, X., Liu, Z., Liu, D., & Ren, X. (2022). A novel CNN-based Bi-LSTM parallel model with attention mechanism for human activity recognition with noisy data. *Sci. Rep.*
14. Zhang, L., Wu, X., & Luo, D. (2015). Human activity recognition with HMM-DNN model. In *2015 IEEE 14th International Conference on Cognitive Informatics & Cognitive Computing (ICCI/CIC)\**, Beijing, China, 192–197.
15. Zhang, M., & Sawchuk, A. A. (2012). USC-HAD: A daily activity dataset for ubiquitous activity recognition using wearable sensors. *ACM*.

## GLOSSARY

1. **Human Activity Recognition (HAR)**: The process of automatically identifying human activities using data collected from sensors.
2. **Wearable Sensors**: Electronic devices that are worn on the body and used to collect data about a person's movements, physical activities, and other physiological parameters.
3. **Deep Learning (DL)**: A subset of machine learning involving neural networks with many layers, capable of learning from large amounts of data.
4. **Convolutional Neural Network (CNN)**: A type of deep neural network particularly effective for analyzing visual data, characterized by convolutional layers that automatically and adaptively learn spatial hierarchies of features.
5. **Recurrent Neural Network (RNN)**: A type of neural network where connections between nodes can create cycles, allowing it to maintain a memory of previous inputs, making it suitable for sequential data.
6. **Long Short-Term Memory (LSTM)**: A special kind of RNN capable of learning long-term dependencies, effectively used for time series prediction.
7. **Gated Recurrent Unit (GRU)**: A variant of the RNN that is simpler than LSTM but can also capture long-term dependencies in sequential data.
8. **Bi-Directional LSTM (Bi-LSTM)**: An extension of LSTM that processes data in both forward and backward directions, improving the learning of context for sequential data.
9. **Attention Mechanism**: A component of neural networks that enables the model to focus on specific parts of the input sequence when making predictions, improving performance on tasks like translation and sequence classification.
10. **Multichannel CNN-GRU Model**: A model combining CNNs for feature extraction and GRUs for temporal sequence processing, often used in HAR for handling multi-sensor data.
11. **Ensemble Learning**: The technique of combining predictions from multiple models to improve overall performance.
12. **Sensor Fusion**: The process of integrating data from multiple sensors to produce more consistent, accurate, and useful information.
13. **Inertial Measurement Unit (IMU)**: A device that measures force, angular rate, and sometimes the magnetic field surrounding the body, often used in wearable sensors for HAR.
14. **Accelerometer**: A sensor that measures the acceleration forces acting on an object, used in HAR to detect movements.
15. **Gyroscope**: A sensor that measures the rate of rotation around an axis, used in HAR to track orientation and angular velocity.
16. **Benchmark Dataset**: A standard dataset used to evaluate and compare the performance of different algorithms and models.
17. **Domain Shift**: The difference in data distributions between training and testing datasets, which can affect the performance of machine learning models.
18. **Data Augmentation**: Techniques used to increase the diversity of training data without collecting new data, such as transformations like rotation, flipping, and scaling.
19. **Generative Adversarial Networks (GANs)**: A type of neural network architecture used to generate synthetic data by training two networks in opposition to each other.
20. **Mean Average Precision (mAP)**: A performance metric used to evaluate object detection models, representing the average precision across all classes.
21. **Morphological Operations**: Image processing operations that process images based on shapes, commonly used in tasks like edge detection and noise removal.
22. **Threat Image Projection (TIP)**: A technique for generating synthetic X-ray images by superimposing images of prohibited items onto benign images.
23. **Embedded Hidden Markov Models (HMMs)**: Statistical models used for temporal pattern recognition, often embedded in larger systems for tasks like HAR.
24. **Spatiotemporal Features**: Features that capture both spatial and temporal aspects of data, crucial for recognizing activities over time and space.
25. **Entropy Markov Model**: A statistical model that incorporates entropy measures into Markov models to capture the uncertainty and variability in sequences.
26. **K-Ary Tree Hashing Classifier**: A classifier that uses a tree-based hashing mechanism to efficiently handle multi-class classification tasks in HAR.



**A REPORT  
ON  
RESOURCE-CONSTRAINED IMPLEMENTATIONS  
OF AI ALGORITHMS  
(POST-MIDSEM WORK DONE)**

**BY**

**RAJAT SRIVASTAVA**

**2022B1A81375G**

**SHIVAM RAJ**

**2022A8PS0397P**

**AT**

**CSIR-CEERI PILANI**

**A Practice School-I Station of**

**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI**

**(JULY,2024)**

## LITERATURE REVIEW:

# “An improved human activity recognition technique based on convolutional neural network”

### Introduction:

Human Activity Recognition (HAR) has become an increasingly important research area within the fields of artificial intelligence and machine learning, with applications ranging from medical diagnosis to surveillance and human-computer interaction. The use of Convolutional Neural Networks (CNNs) in HAR is particularly significant due to their ability to automatically extract and learn complex features from raw sensor data, thus improving the accuracy of activity classification tasks.

### Dataset Overview and Rationale:

The study utilizes the WISDM (Wireless Sensor Data Mining) dataset, which is publicly available and widely used for HAR tasks. This dataset is chosen because it contains a large number of labeled sequences of human activities, collected using accelerometer sensors embedded in smartphones. The WISDM dataset is well-suited for developing and evaluating CNN models due to its comprehensive nature and the variety of activities it includes.

### Data Acquisition and Preprocessing:

Data acquisition for the WISDM dataset involves collecting accelerometer readings from smartphones carried by participants performing different activities. The raw accelerometer data is segmented into fixed-size windows, which are then used as input for the CNN model. Preprocessing steps include normalization of the accelerometer readings to ensure consistency and to improve the model's performance. Additionally, data augmentation techniques are applied to increase the diversity of the training data and to prevent overfitting.

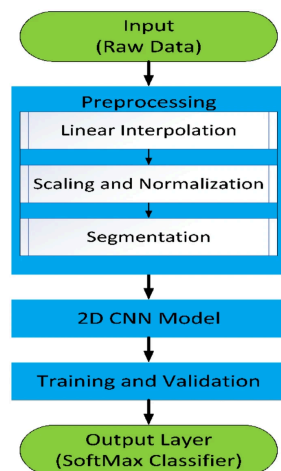
### Architecture:

The proposed HAR technique is based on a multi-layered CNN architecture. The CNN model consists of several convolutional layers followed by pooling layers and fully connected layers. Convolutional layers are responsible for extracting spatial features from the input data, while pooling layers reduce the spatial dimensions, thus helping in reducing the computational load. The fully connected layers at the end of the network aggregate the extracted features and perform the final classification of activities.

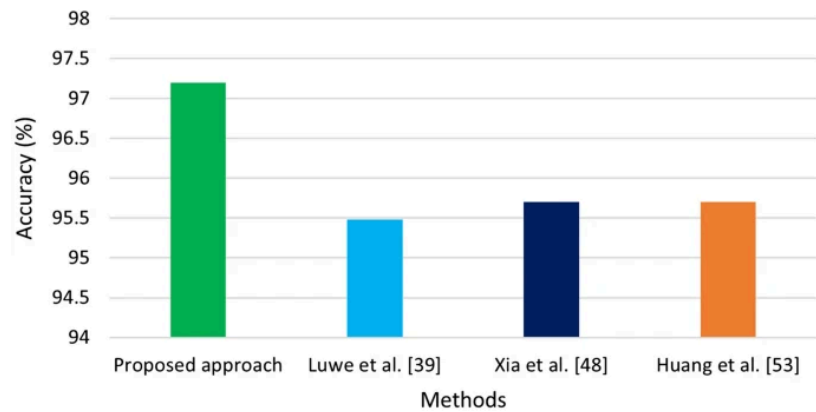
Key components of the CNN architecture include:

- **Convolutional Layers:** These layers apply convolution operations using multiple filters to extract local features from the input data.
- **Pooling Layers:** Max pooling is used to down-sample the feature maps, retaining the most significant features.
- **Fully Connected Layers:** These layers perform the final classification based on the features

extracted by the convolutional and pooling layers



Accuracy Analysis

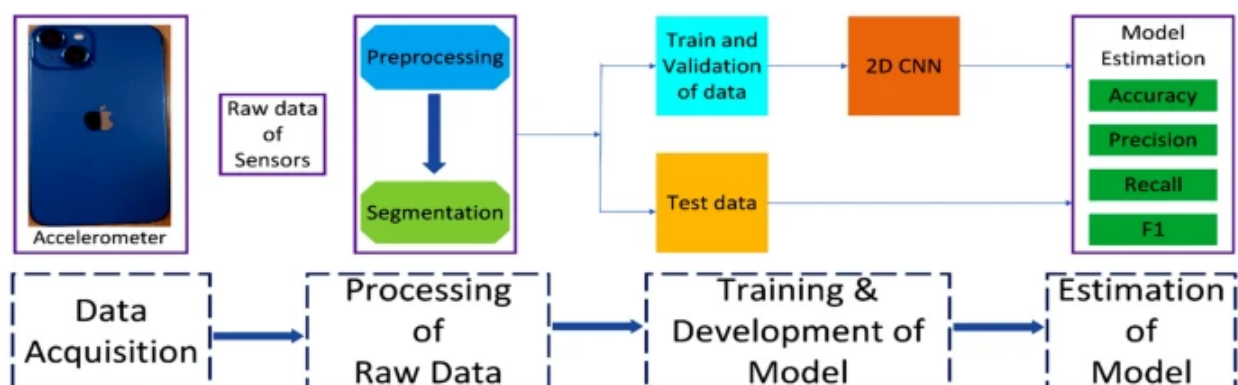


## Experimental Results:

The proposed CNN-based HAR model achieves a high accuracy rate of 97.20%, outperforming previous state-of-the-art techniques. This result demonstrates the effectiveness of CNNs in capturing both spatial and temporal features from accelerometer data. The model's performance is evaluated using standard metrics such as accuracy, precision, recall, and F1-score, indicating its robustness and reliability in real-world scenarios.

## Method of Working:

The CNN model works by taking segmented accelerometer data as input and passing it through multiple layers of convolutional and pooling operations. The convolutional layers detect patterns and features relevant to different activities, while the pooling layers help in reducing the dimensionality of the feature maps. The final fully connected layers aggregate these features and make predictions about the activity being performed.



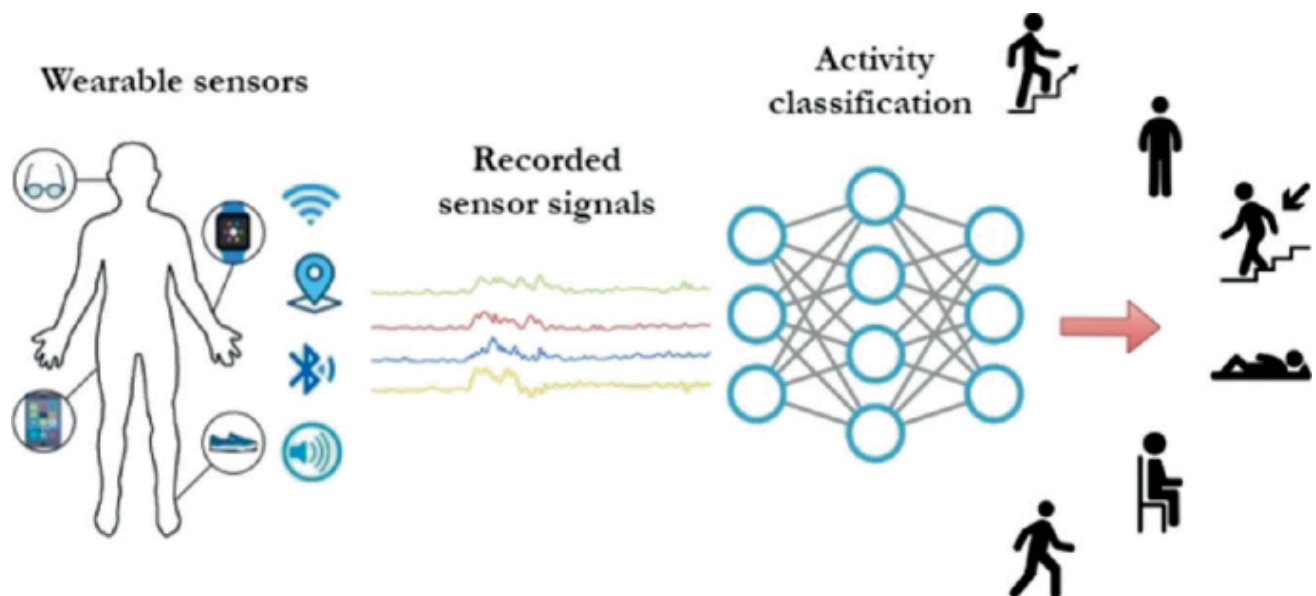
The framework of human activity recognition.

### Compatibility:

The proposed HAR technique is implemented using Python and standard deep learning libraries such as TensorFlow and Keras. This ensures compatibility with various hardware and software environments. The use of Python allows for easy integration with other data processing and machine learning tools, making the model accessible for further research and practical applications.

### Conclusion and Future Perspectives:

The study concludes that CNNs are highly effective for HAR tasks, significantly improving the accuracy of activity classification. Future research directions include exploring the use of other deep learning architectures, such as recurrent neural networks (RNNs) and long short-term memory (LSTM) networks, for better capturing temporal dependencies in sensor data. Additionally, expanding the dataset with more diverse activities and incorporating data from other types of sensors could further enhance the model's performance and applicability.



# READING ON DATA SETS: WISDM DATASET

## Dataset Description and Model Training:

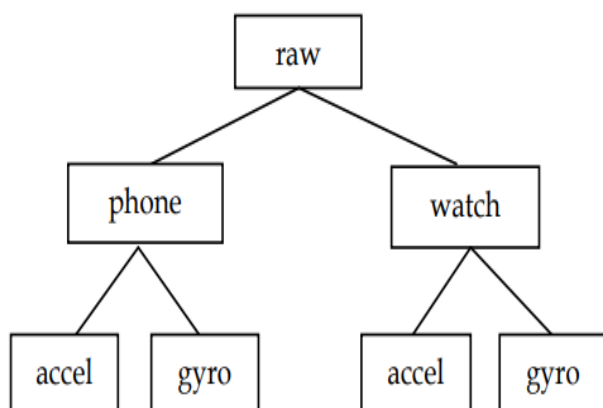
In this project, we utilized the "WISDM Smartphone and Smartwatch Activity and Biometrics Dataset" to train our 1D Convolutional Neural Network (CNN) model for Human Activity Recognition (HAR). The dataset, collected by the Wireless Sensor Data Mining (WISDM) Lab at Fordham University, includes data from 51 subjects performing 18 different activities of daily living. The activities include ambulation-related actions such as walking, jogging, and climbing stairs, as well as hand-based activities like brushing teeth, folding clothes, and various eating activities.

The dataset comprises time-series sensor data from the accelerometer and gyroscope of a smartphone and a smartwatch, collected at a rate of 20 Hz. This high-frequency data collection enabled us to capture detailed motion patterns. Each activity was performed for 3 minutes per subject, resulting in a substantial amount of data for each activity.

The raw sensor data was pre-processed using a sliding window approach, transforming the time-series data into labeled examples suitable for training machine learning models. Our 1D CNN model was trained to classify six specific activities: walking, running, upstairs, downstairs, sitting, and standing. The transformed data facilitated the effective training and evaluation of our model, enabling accurate activity recognition based on the sensor inputs.

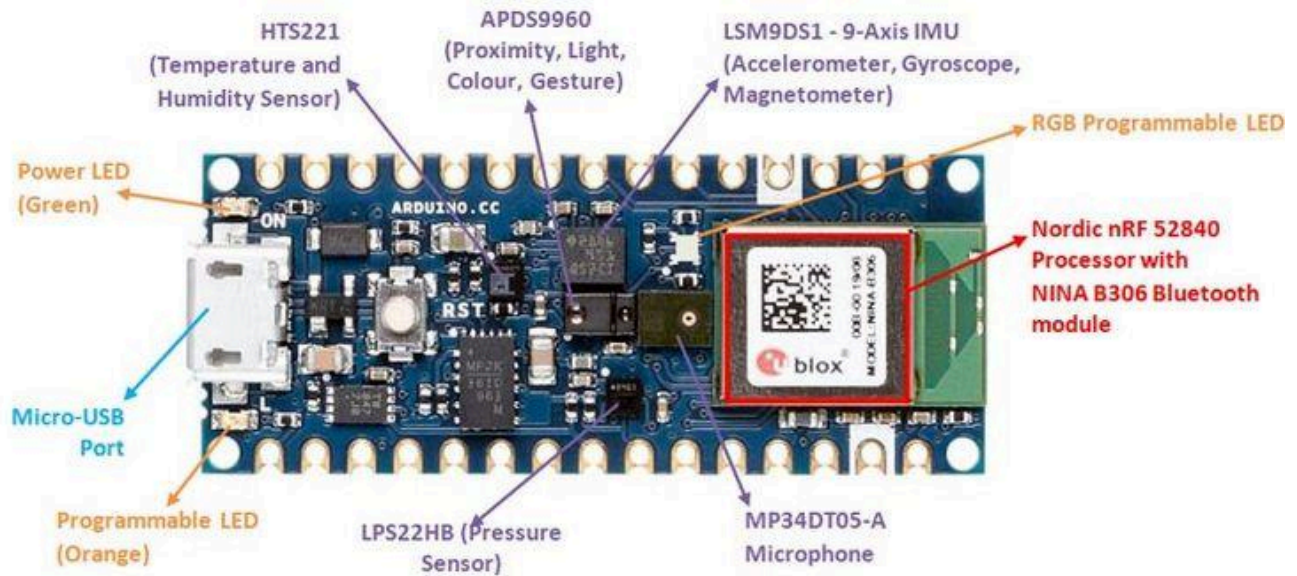
TABLE 4  
DISTRIBUTION OF RAW SENSOR DATA

| Activity     | Phone     |           | Watch     |           | Total      | Class % |
|--------------|-----------|-----------|-----------|-----------|------------|---------|
|              | Accel     | Gyro      | Accel     | Gyro      |            |         |
| Walking      | 279,817   | 203,919   | 210,495   | 192,531   | 886,762    | 5.7%    |
| Jogging      | 268,409   | 200,252   | 205,787   | 187,833   | 862,281    | 5.5%    |
| Stairs       | 255,645   | 197,857   | 207,312   | 180,416   | 841,230    | 5.4%    |
| Sitting      | 264,592   | 202,370   | 213,018   | 195,050   | 875,030    | 5.6%    |
| Standing     | 269,604   | 202,351   | 216,529   | 194,103   | 882,587    | 5.6%    |
| Typing       | 246,356   | 194,540   | 205,137   | 187,175   | 833,208    | 5.3%    |
| Brush Teeth  | 269,609   | 202,622   | 208,720   | 190,759   | 871,710    | 5.6%    |
| Eat Soup     | 270,756   | 202,408   | 209,483   | 187,057   | 869,704    | 5.6%    |
| Eat Chips    | 261,360   | 197,905   | 210,048   | 192,085   | 861,398    | 5.5%    |
| Eat Pasta    | 249,793   | 197,844   | 203,112   | 189,609   | 840,358    | 5.4%    |
| Drinking     | 285,190   | 202,395   | 215,879   | 197,917   | 901,381    | 5.8%    |
| Eat Sandwich | 265,781   | 197,915   | 203,684   | 190,191   | 857,571    | 5.5%    |
| Kicking      | 278,766   | 202,625   | 209,491   | 191,535   | 882,417    | 5.6%    |
| Catch        | 272,219   | 198,756   | 210,107   | 187,684   | 868,766    | 5.6%    |
| Dribbling    | 272,730   | 202,331   | 212,810   | 194,845   | 882,716    | 5.6%    |
| Writing      | 260,497   | 197,894   | 215,365   | 197,403   | 871,159    | 5.6%    |
| Clapping     | 268,065   | 202,330   | 208,734   | 190,776   | 869,905    | 5.6%    |
| Fold Clothes | 265,214   | 202,321   | 211,335   | 193,373   | 872,243    | 5.6%    |
| Total        | 4,804,403 | 3,608,635 | 3,777,046 | 3,440,342 | 15,630,426 | 100%    |



# Implementing HAR Algorithm on 1D CNN vs 2D CNN on:

## Arduino Nano 33 BLE Sense Nano



The Arduino Nano 33 BLE Sense is a powerful microcontroller board that is well-suited for implementing machine learning models, particularly in resource-constrained environments. For our Human Activity Recognition (HAR) project, we have chosen to deploy a 1D Convolutional Neural Network (CNN) model on this board using the Arduino IDE.

### Reasons for Choosing Arduino Nano 33 BLE Sense

#### 1. Compact and Versatile Hardware:

- The Arduino Nano 33 BLE Sense is a compact board equipped with a range of high-accuracy sensors, including an accelerometer, gyroscope, magnetometer, temperature, humidity, pressure, and light sensors. This makes it ideal for capturing the diverse range of data needed for HAR tasks with high precision.

#### 2. Low Power Consumption:

- The board is designed to operate with low power consumption, which is crucial for battery-operated applications. This ensures longer operational time and reliability in portable and wearable devices.

#### 3. Built-in Bluetooth Low Energy (BLE):

- The inclusion of BLE facilitates easy wireless communication, allowing the board to transmit sensor data to other devices, such as smartphones or laptops, for further processing or visualization.

#### 4. Suitable for Resource-Constrained Applications:

- The Arduino Nano 33 BLE Sense is designed for applications with limited memory and processing power. Our 1D CNN model, optimized to be less than 1 MB, fits well within the board's constraints, ensuring smooth and efficient operation.

**5. Ease of Use with Arduino IDE:**

- The Arduino IDE provides an accessible and user-friendly environment for programming and deploying machine learning models. It supports a wide range of libraries and tools, including TensorFlow Lite for Microcontrollers, which simplifies the process of running machine learning models on microcontroller boards.

**6. High Accuracy of Sensors:**

- The sensors on the Arduino Nano 33 BLE Sense are known for their high accuracy, which is essential for reliable HAR tasks. Accurate sensor data improves the model's ability to correctly classify human activities, thereby enhancing overall performance.

**7. Strong Community and Support:**

- The Arduino ecosystem has a large, active community and extensive documentation, providing valuable resources and support for troubleshooting and development. This helps accelerate the development process and enables quick resolution of any issues that arise.

## 1D CNN vs 2D CNN:

Human Activity Recognition (HAR) involves classifying physical activities using sensor data, and Convolutional Neural Networks (CNNs) are commonly used for this purpose. While 2D CNNs have been traditionally used for image-related tasks, 1D CNNs are more suited for time-series data such as accelerometer readings. In resource-constrained environments, like the Arduino Nano BLE Sense, model size becomes a critical factor. This comparison focuses on the implementation of a 1D CNN over a 2D CNN due to its significantly smaller size, making it suitable for deployment on devices with limited memory and computational resources.

### Dataset Overview:

For this comparison, we use the WISDM (Wireless Sensor Data Mining) dataset, which provides a comprehensive collection of accelerometer data recorded from smartphones. The dataset includes a variety of activities such as walking, jogging, climbing stairs, and sitting, making it an ideal benchmark for HAR tasks.

### 1D CNN Model for HAR:

1D CNNs are designed to process sequential data, making them well-suited for time-series data such as accelerometer readings. The architecture typically includes several convolutional layers followed by pooling layers and fully connected layers.

- **Convolutional Layers:** These layers apply 1D convolutions along the temporal dimension, capturing local patterns in the sensor data.
- **Pooling Layers:** Max pooling is used to reduce the temporal dimensions, retaining the most significant features.
- **Fully Connected Layers:** These layers aggregate the features extracted by the convolutional layers and perform the final classification.

### Advantages of 1D CNN for HAR:

- **Temporal Feature Extraction:** 1D convolutions effectively capture temporal dependencies in the data.
- **Efficiency:** 1D CNNs require fewer parameters and computational resources compared to 2D CNNs.
- **Compact Model Size:** The 1D CNN model designed for this study is significantly smaller, with a size optimized to be less than 1 MB, making it suitable for deployment on resource-constrained devices like the Arduino Nano BLE Sense.

### 2D CNN Model for HAR:

2D CNNs are typically used for image data but can be applied to time-series data by transforming the input into a 2D format. This involves reshaping the sensor data into a 2D matrix, where one



dimension represents time and the other represents the different sensor channels.

- **Convolutional Layers:** These layers apply 2D convolutions to the transformed data, capturing spatial patterns.
- **Pooling Layers:** Max pooling reduces the spatial dimensions, preserving important features.
- **Fully Connected Layers:** These layers perform the final classification based on the extracted features.

### **Disadvantages of 2D CNN for HAR in Resource-Constrained Environments:**

- **Larger Model Size:** 2D CNN models tend to be larger due to the higher number of parameters required for 2D convolutions. The model size often exceeds 15 MB, which is unsuitable for devices with limited memory.
- **Higher Computational Requirements:** 2D CNNs demand more computational power, which can be a limiting factor for devices like the Arduino Nano BLE Sense.

### **Experimental Results**

The 1D CNN model achieved an accuracy close to 94%, demonstrating its effectiveness in capturing temporal features from the accelerometer data. More importantly, the model size was optimized to be less than 1 MB, making it compatible with the memory constraints of the Arduino Nano BLE Sense.

### **Performance Metrics for 1D CNN:**

- **Accuracy:** Close to 94%
- **Model Size:** <1 MB

### **Method of Working**

The workflow for both models involves:

1. **Data Segmentation:** Raw sensor data is segmented into fixed-size windows.
2. **Normalization:** Data is normalized to ensure consistency.
3. **Model Training:** The segmented data is used to train the CNN models.
4. **Evaluation:** The models are evaluated using standard metrics to determine their performance.

### **Compatibility**

The 1D CNN model's compact size and lower computational requirements make it highly compatible with resource-constrained devices like the Arduino Nano BLE Sense. Implemented using Python and deep learning libraries such as TensorFlow Lite, the model ensures compatibility with various hardware and software environments, facilitating deployment on edge devices.

### **Conclusion**

The main reason for implementing our HAR algorithm using a 1D CNN instead of a 2D CNN is the significant reduction in model size. The 1D CNN model, optimized to be less than 1 MB, fits well

within the memory constraints of the Arduino Nano BLE Sense. This makes the 1D CNN model not only efficient in terms of performance but also practical for deployment in real-world, resource-constrained environments. Future research could focus on further optimizing model architectures and exploring hybrid models to balance size and performance.

## IMPLEMENTATION:

### 1. Model Training in Python

**Architecture Brief:** We designed a 1D Convolutional Neural Network (CNN) for Human Activity Recognition (HAR) using accelerometer data. The model architecture includes:

- Two convolutional layers with ReLU activation and max-pooling layers to capture temporal features.
- A flatten layer to convert the 3D tensor output from the convolutional layers to a 1D vector.
- A dense layer with ReLU activation and dropout for regularization.
- An output layer with softmax activation to classify six activities: walking, jogging, sitting, standing, downstairs, and upstairs.

The purpose of this architecture is to create a lightweight model that can effectively capture the temporal dynamics of human activities from accelerometer data while remaining small enough to run on resource-constrained devices like the Arduino Nano BLE Sense. By using fewer filters and a smaller number of units in the dense layer, we ensure the model remains compact and efficient.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Dropout, Flatten, Dense
from tensorflow.keras.optimizers import Adam

model = Sequential()
model.add(Conv1D(16, 2, activation='relu', input_shape=(80, 3)))
model.add(MaxPooling1D(2))
model.add(Conv1D(24, 2, activation='relu'))
model.add(MaxPooling1D(2))
model.add(Flatten())
model.add(Dense(48, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(6, activation='softmax'))
model.compile(optimizer=Adam(learning_rate=0.001), loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=75, validation_data=(X_test, y_test), verbose=1)
```

### 2. Model Conversion to TensorFlow Lite

**Purpose:** Converting the trained model to TensorFlow Lite format optimizes it for deployment on microcontrollers by reducing its size and computational requirements. This step involves transforming the full TensorFlow model into a format that is suitable for inference on devices with limited resources. TensorFlow Lite is designed to be lightweight and fast, making it ideal for running

machine learning models on microcontrollers. This conversion process ensures that our model can fit within the memory constraints of the Arduino Nano BLE Sense while maintaining sufficient accuracy for real-time HAR tasks. By using TensorFlow Lite, we can achieve efficient model execution with minimal latency and power consumption.

```
import tensorflow as tf

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
open("HAR.tflite", "wb").write(tflite_model)
```

### 3. TensorFlow Lite Model Conversion to Header File

**Purpose:** Converting the TensorFlow Lite model to a C array format enables us to embed the model directly into the Arduino sketch. This step is essential for integrating the model into the Arduino development environment, allowing it to be used within the C++ codebase that runs on the microcontroller.

The conversion to a header file format allows us to include the model as a constant byte array in our Arduino project. This approach simplifies the deployment process and ensures that the model is readily accessible during program execution. By embedding the model in this way, we can leverage the Arduino IDE to manage the model alongside other project files, facilitating seamless integration and deployment.

```
import os

tflite_file = "HAR.tflite"
with open(tflite_file, 'rb') as f:
    tflite_model = f.read()
c_array = ', '.join(map(lambda x: f"0x{x:02x}", tflite_model))

header_file_path = "model.h"
with open(header_file_path, 'w') as f:
    f.write("const unsigned char model[] = {\n")
    f.write(c_array)
    f.write("\n};\n")

model_h_size = os.path.getsize(header_file_path)
print(f"Header file, {header_file_path}, is {model_h_size:,} bytes.")
```

### 4. Deploying the Model on Arduino Nano Using Arduino IDE

**Purpose:** Deploying the model on the Arduino Nano BLE Sense involves writing C++ code in the Arduino IDE to run the inference on real-time data. This step requires initializing the model, setting up the sensors, and implementing the logic to capture and process accelerometer data.

The Arduino IDE provides a convenient platform for programming and uploading code to the Arduino Nano BLE Sense. By utilizing the IDE, we can write and debug C++ code that interfaces with the TensorFlow Lite model and the board's sensors. This deployment process allows us to run the HAR model on the microcontroller, enabling real-time activity recognition using the built-in accelerometer.

```
#include <Arduino_LSM9DS1.h>
#include <ArduTFLite.h>
#include "model.h"

const int numSamples = 80;
int samplesRead = 0;
const int inputLength = 240;
constexpr int tensorArenaSize = 16 * 1024;
alignas(16) byte tensorArena[tensorArenaSize];

const char* GESTURES[] = {"Downstairs", "Jogging", "Sitting", "Standing", "Upstairs", "Wall"};
#define NUM_GESTURES (sizeof(GESTURES) / sizeof(GESTURES[0]))
```

```
void setup() {
    Serial.begin(9600);
    while (!Serial);

    if (!IMU.begin()) {
        Serial.println("IMU sensor init failed!");
        while (true);
    }

    Serial.print("Accelerometer sampling frequency = ");
    Serial.print(IMU.accelerationSampleRate());
    Serial.println(" Hz");

    Serial.println("Init model...");
    if (!modelInit(model, tensorArena, tensorArenaSize)) {
        Serial.println("Model initialization failed!");
        while (true);
    }
    Serial.println("Model initialization done.");
}
```

```

void loop() {
  float x, y, z;
  while (samplesRead < numSamples) {
    if (IMU.accelerationAvailable()) {
      IMU.readAcceleration(x, y, z);
      modelSetInput(x, samplesRead * 3 + 0);
      modelSetInput(y, samplesRead * 3 + 1);
      modelSetInput(z, samplesRead * 3 + 2);

      samplesRead++;
      if (samplesRead == numSamples) {
        if (!modelRunInference()) {
          Serial.println("RunInference Failed!");
          return;
        }

        for (int i = 0; i < NUM_GESTURES; i++) {
          Serial.print(GESTURES[i]);
          Serial.print(": ");
          Serial.print(modelGetOutput(i) * 100, 2);
          Serial.println("%");
        }
        Serial.println();
        samplesRead = 0;
      }
    }
  }
}

```

## Using IMU for Real-Time Analysis of HAR

**Purpose:** The Inertial Measurement Unit (IMU) on the Arduino Nano BLE Sense captures real-time accelerometer data, which is fed into the 1D CNN model for activity classification. This step is crucial for performing live HAR, as it involves continuously collecting sensor data and making predictions based on the model's output.

The IMU provides high-accuracy sensor readings, which are essential for reliable activity recognition. By processing the accelerometer data through the TensorFlow Lite model, we can classify activities with high precision. The results are displayed in real-time on the serial monitor, showing the likelihood of each activity as percentages. This real-time analysis allows us to monitor and evaluate the performance of the HAR system directly on the Arduino Nano BLE Sense.

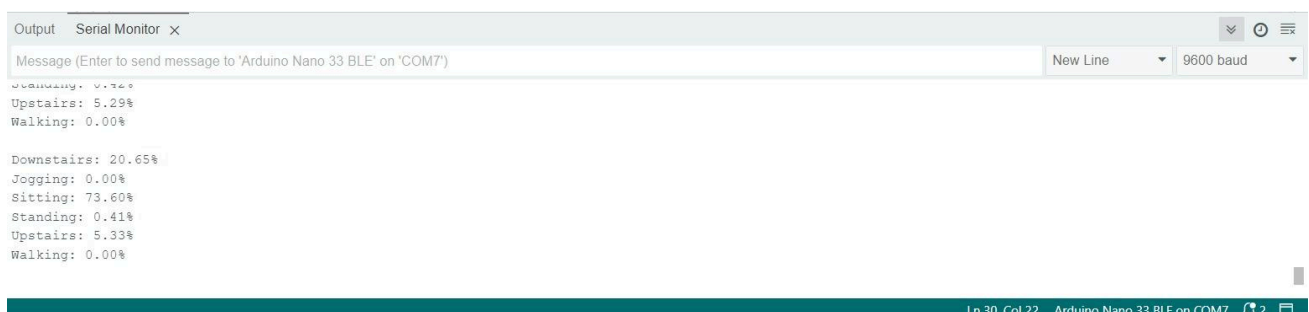
## TensorFlow Lite Libraries:

The TensorFlow Lite libraries provide essential functions for loading, interpreting, and running machine learning models on microcontrollers. These libraries are optimized for resource-constrained devices, enabling efficient model execution with minimal computational overhead. Using TensorFlow Lite, we can leverage advanced machine learning capabilities on the Arduino Nano BLE Sense, transforming it into a powerful tool for real-time HAR.

## Conclusion and Results:

The implementation of our 1D CNN model on the Arduino Nano BLE Sense yielded highly satisfactory results. Key outcomes include:

- **Model Size:** The final TensorFlow Lite model size was approximately 600KB, which is well within the memory constraints of the Arduino Nano BLE Sense.
- **Accuracy:** The model achieved an accuracy close to 92-94% on the test dataset, indicating strong performance in classifying the six activities.
- **Real-Time Performance:** The model was able to process and classify accelerometer data in real-time, with classification results displayed as percentages on the serial monitor. This real-time performance is critical for practical applications of HAR in wearable and IoT devices.
- **Sensor Accuracy:** The high accuracy of the Arduino Nano BLE Sense's built-in sensors contributed significantly to the reliable performance of the HAR model. The precise accelerometer readings ensured that the model received high-quality input data for accurate activity classification.



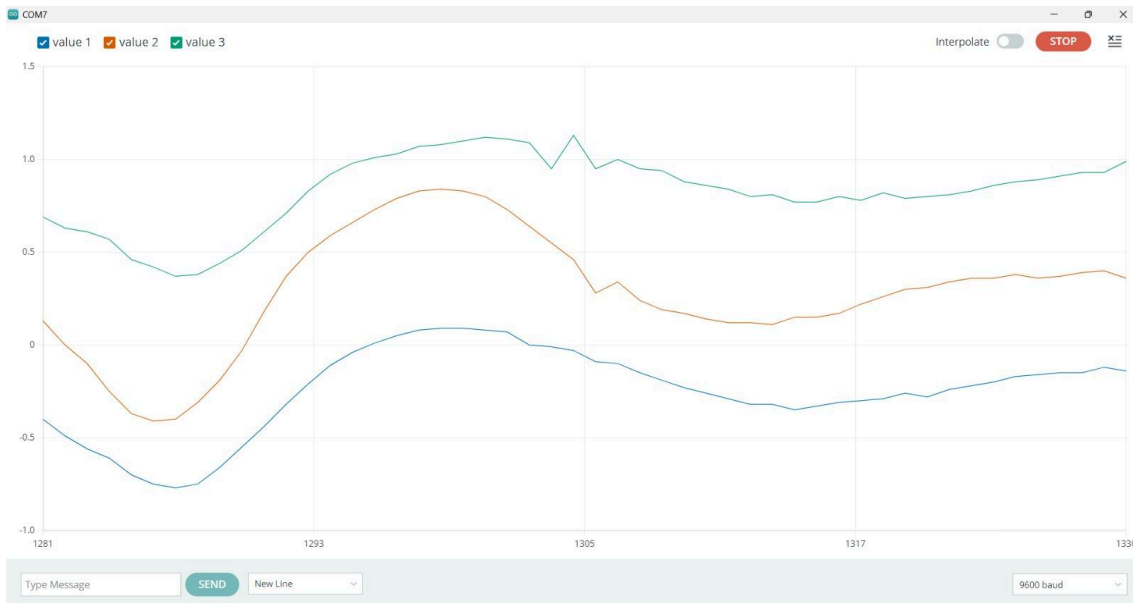
The screenshot shows the Arduino IDE Serial Monitor window. The title bar reads 'Output Serial Monitor x'. The message input field contains 'Message (Enter to send message to 'Arduino Nano 33 BLE' on 'COM7')'. The dropdown menus show 'New Line' and '9600 baud'. The serial output displays the following data:

```
Summary: v.1.0.0
Upstairs: 5.29%
Walking: 0.00%

Downstairs: 20.65%
Jogging: 0.00%
Sitting: 73.60%
Standing: 0.41%
Upstairs: 5.33%
Walking: 0.00%
```

The status bar at the bottom indicates 'Ln 30, Col 22 Arduino Nano 33 BLE on COM7'.

By following these steps, we successfully trained a 1D CNN model, converted it to TensorFlow Lite format, and deployed it on the Arduino Nano BLE Sense using the Arduino IDE. This allows us to perform real-time Human Activity Recognition with high accuracy, leveraging the board's sensors and the power of machine learning on a resource-constrained device. The entire process demonstrates the feasibility and effectiveness of deploying advanced machine learning models on microcontrollers for practical applications.



Serial Monitor Representation of Accelerometer Data.

## Appendices:

### Appendix A: WISDM Dataset Details

- Detailed description of the WISDM dataset
- Data collection methodology
- List of activities included in the dataset
- Sensor specifications

### Appendix B: Model Architecture Diagram

- Visual representation of the 1D CNN architecture used in the study

### Appendix C: Performance Metrics

- Detailed tables of accuracy, precision, recall, and F1-score for each activity

## References:

1. Warden, P., & Situnayake, D. (2019). TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media.
2. TensorFlow Lite Documentation. (n.d.). TensorFlow. Retrieved from <https://www.tensorflow.org/lite>
3. Arduino. (n.d.). Arduino Nano 33 BLE Sense. Retrieved from <https://docs.arduino.cc/hardware/nano-33-ble-sense>
4. Kwapisz, J. R., Weiss, G. M., & Moore, S. A. (2011). Activity recognition using cell phone accelerometers. ACM SigKDD Explorations Newsletter, 12(2), 74-82.
5. TensorFlow. (n.d.). Gesture recognition with Arduino. Retrieved from [https://www.tensorflow.org/lite/microcontrollers/get\\_started\\_low\\_level#gesture\\_recognition\\_with\\_arduino](https://www.tensorflow.org/lite/microcontrollers/get_started_low_level#gesture_recognition_with_arduino)

## Glossary:

1. CNN (Convolutional Neural Network): A type of deep learning model particularly effective for processing grid-like data, such as images or time series.
2. HAR (Human Activity Recognition): The process of automatically identifying and classifying human activities based on sensor data.
3. TinyML: The field of machine learning applied to extremely low-power systems, typically running on microcontrollers.
4. TensorFlow Lite: A lightweight version of TensorFlow designed for mobile and embedded devices.
5. WISDM (Wireless Sensor Data Mining): Refers to the dataset used in this study, collected by the WISDM Lab at Fordham University.
6. IMU (Inertial Measurement Unit): A device that measures and reports specific force, angular rate, and sometimes the orientation of the body, using a combination of accelerometers, gyroscopes, and sometimes magnetometers.
7. Arduino IDE: The Integrated Development Environment used for programming Arduino boards.
8. BLE (Bluetooth Low Energy): A wireless personal area network technology designed for novel applications in the healthcare, fitness, security, and home entertainment industries.
9. Accelerometer: A sensor that measures proper acceleration (g-force), commonly used in mobile devices and wearables for motion sensing.
10. Gyroscope: A sensor that measures angular velocity, often used in combination with accelerometers for more accurate motion tracking.